REVA UNIVERSITY
Bengaluru, India

SCHOOL OF COMPUTER SCIENCE AND APPLICATIONS

Minor Project Report

on

EXPLAINABLE AI FOR BRAIN TUMOUR DIAGNOSIS FROM MEDICAL IMAGING

A Project Report submitted in partial fulfilment of the requirements for the award of the Degree of Master of Science in Data Science - M.Sc. (DS)

Submitted by

Hemanth Gowda H L -R24DG010

Under the Guidance of

Dr. P SREE LAKSHMI

DECEMBER 2025

Rukmini Knowledge Park, Kattigenahalli, Yelahanka, Bengaluru-560064
www.reva.edu.in

CERTIFICATE

This is to certify that the Minor project work entitled "**Explainable AI For Brain Tumour Diagnosis From Medical Imaging**" submitted to the School of Computer Science and Applications, REVA University in partial fulfilment of the requirements for the award of the Degree of **Master of Science in Data Science** in the academic year 2024-2025 is a record of the original work done by **Hemanth Gowda H L (R24DG010)** under my supervision and guidance. The project report has been approved as it satisfies the academic requirements in respect of Semester III Project work prescribed for the said Degree and this Minor project work has not formed the basis for the award of any Degree / Diploma / Associate ship / Fellowship or similar title to any candidate of any University.

Signature with date **Signature with date**

Dr. P Sree Lakshmi
Internal Guide

Dr. Ambili P S
Incharge Director

Name of the Examiner with affiliation Signature with Date

1.

2.

DECLARATION

I, **Hemanth Gowda H L (R24DG010)** and **Dimanth Charles (R24DG006)** third semester students of Master of Science in Data Science belonging to School of Computer Science and Applications, REVA University, declare that this Project work entitled **"Explainable AI For Brain Tumour Diagnosis From Medical Imaging"** is the result of the Project work done by us under the supervision of **Dr. P Sree Lakshmi**.

I submitting this Project work in partial fulfilment of the requirements for the award of the degree of Master of Science in Data Science by REVA University, Bangalore during the academic year 2024-25.

I further declare that this Project report or any part of it has not been submitted for the award of any other Degree / Diploma of this University or any other University / Institution.

Signed on:

*Certified that this project work submitted by **Hemanth Gowda H L** has been carried out under my guidance and the declaration made by the candidates is true to the best of my knowledge.*

Signature of the Guide
Date:

Signature of the Incharge Director,
Date:

ACKNOWLEDGEMENT

I hereby acknowledge all those, under whose support and encouragement, we have been able to complete these academic commitments successfully. In this regard, we take this opportunity to express our deep sense of gratitude and sincere thanks to School of Computer Science and Applications which has always been a tremendous source of guidance.

I express our sincere gratitude to **Dr. P. SHYAMA RAJU**, Honourable Chancellor, REVA University, Bengaluru for providing us the state-of-the-art facilities.

I thankful to **Dr. SANJAY CHITNIS**, Vice Chancellor, REVA University, **Dr. M.DHANAMJAYA**, Registrar, REVA University, for their support and encouragement.

I take this opportunity to express our heartfelt sincere thanks to **Dr. LOKESH C.K.**, Director, School of CSA and **Dr. AMBILI P S**, Incharge Director - PG, School of CSA, REVA University for the encouragement and best wishes provided impetus for the Project Work carried out.

I convey warm and sincere gratitude to our guide **Dr. P Sree Lakshmi** for the valuable suggestion and constant encouragement towards the completion of this project.

Last, but not the least, we express our gratitude to everyone who provided support and encouragement during the course of the project.

Hemanth Gowda H L (R24DG010)

ABSTRACT

Brain tumours are among the most serious neurological disorders and require early and accurate diagnosis to improve treatment outcomes and patient survival rates. Magnetic Resonance Imaging (MRI) is widely used for detecting brain tumours because it provides high-resolution and detailed images of brain tissues. However, manual examination of MRI scans is a time-consuming process and depends greatly on the experience of radiologists, which can sometimes lead to delayed diagnosis. To overcome these challenges, this project proposes an automated brain tumour prediction system using deep learning techniques. The system employs EfficientNetB0, a pre-trained convolutional neural network model known for its efficiency and high classification accuracy, to analyze MRI images and classify them into tumour and non-tumour categories. In addition to accurate prediction, the model incorporates Explainable Artificial Intelligence techniques such as Gradient-weighted Class Activation Mapping (Grad-CAM) to provide visual explanations by highlighting important regions in MRI images that influence the model's decision, thereby helping to identify tumour-affected areas. The proposed system achieves reliable performance with high accuracy and improved interpretability, making it a valuable decision-support tool for medical professionals. While it does not replace clinical diagnosis, it assists radiologists by providing faster, consistent, and more accurate analysis of brain MRI scans.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 INTRODUCTION TO PROJECT

Medical image analysis has become an important area of research due to the increasing availability of digital medical data and advancements in artificial intelligence. Among various medical conditions, brain tumours are considered one of the most critical and life-threatening diseases, requiring early and accurate diagnosis for effective treatment. Magnetic Resonance Imaging (MRI) is widely used for brain tumour detection because it provides high-resolution images of soft tissues without exposing patients to harmful radiation.

Manual analysis of brain MRI images by radiologists is a time-consuming and error-prone process, especially when dealing with large volumes of data. To overcome these challenges, automated systems based on deep learning techniques are increasingly being explored. Deep learning models, particularly Convolutional Neural Networks (CNNs), have shown remarkable performance in image classification and medical diagnosis tasks.

This project focuses on the development of an automated brain tumour prediction system using brain MRI images. The system employs the EfficientNetB0 deep learning model to classify MRI images into tumour and non-tumour categories. In addition to classification, the project integrates Grad-CAM (Gradient-weighted Class Activation Mapping) to visually highlight tumour regions and density variations within the MRI image. This helps improve interpretability and trust in the model's predictions.

The proposed system aims to assist medical professionals by providing fast, accurate, and explainable predictions, thereby supporting early diagnosis and decision-making in clinical environments.

1.2 STATEMENT OF THE PROBLEM

Brain tumour diagnosis using MRI images is a complex and challenging task that requires expert knowledge and careful examination. Traditional diagnostic methods rely heavily on the experience of radiologists, which can lead to variability in interpretation and possible diagnostic errors. Additionally, the increasing number of MRI scans generated in hospitals places a significant burden on healthcare professionals.

Existing automated systems often suffer from limitations such as low accuracy, high computational cost, lack of explainability, or poor generalization to unseen data. Many systems

function as “black boxes,” making it difficult for doctors to understand why a particular decision was made.

Therefore, there is a need for an intelligent, accurate, and interpretable system that can automatically detect brain tumours from MRI images and provide visual explanations for its predictions. The problem addressed in this project is to design and implement such a system using an efficient deep learning model and explainable AI techniques.

1.3 SYSTEM SPECIFICATIONS

HARDWARE REQUIRMENT

Component	**Specification**
Processor (CPU)	Intel Core i5 / i7 or AMD Ryzen 5 / 7 (minimum 3.0 GHz)
GPU (Optional but Recommended)	NVIDIA GPU with CUDA support (e.g., RTX 3060 / Tesla / GTX 1660)
RAM	Minimum 8 GB (Recommended 16 GB for faster training)
Storage	Minimum 100 GB free space (SSD preferred for better I/O performance)
Display	1080p resolution or higher for visualization of MRI and heatmaps
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS

SOFTWARE REQUIRMENT

Category	**Specification**
Programming Language	Python 3.8 or above
Deep Learning Frameworks	TensorFlow / Keras (for NASNet Large and model training)
Explainable AI Tools	LIME, Grad-CAM
Data Processing Libraries	NumPy, Pandas, OpenCV, Scikit-learn
Visualization Libraries	Matplotlib, Seaborn
Image Handling	PIL (Python Imaging Library)
Development Environment	Jupyter Notebook / VS Code / PyCharm
Database (Optional)	MongoDB or SQLite (for storing results and metadata)
Version Control	Git / GitHub

2. LITERATURE SURVEY

This chapter reviews existing research on brain tumour detection using MRI images and deep learning techniques. It highlights the strengths and limitations of previous CNN-based and transfer learning models, emphasizing the need for accurate and explainable tumour prediction systems.

[1] Mohamed Musthafa et al. presented a brain tumour detection approach using the ResNet50 deep learning architecture integrated with Gradient-weighted Class Activation Mapping (Grad-CAM). The main objective of this research was to address the lack of transparency in deep learning-based medical diagnosis systems. MRI images were used as input, and data augmentation techniques were applied to improve model generalization. The proposed model achieved a testing accuracy of 98.52%, with high precision and recall values. Grad-CAM was used to generate visual heatmaps highlighting tumour regions in MRI images, thereby improving interpretability and trust among medical professionals. This study demonstrates that combining high accuracy with explainability is essential for real-world clinical applications. proposed an advanced brain tumour detection framework using MRI images combined with deep learning and explainable artificial intelligence techniques. The study utilizes multiple feature extraction methods such as Local Binary Patterns (LBP), Gabor filters, Discrete Wavelet Transform, and Convolutional Neural Networks (CNN) to improve classification accuracy. Various machine learning classifiers including Support Vector Classifier (SVC) and CNN were evaluated. To enhance interpretability, SHAP analysis was employed to identify important features influencing predictions. The results demonstrated that CNN achieved an accuracy of up to 98.9%, showing strong performance in automated tumour detection. This work highlights the importance of combining deep learning with explainable AI to improve diagnostic reliability in medical imaging

[2] Sultan et al.proposed an automated brain tumour classification system using convolutional neural networks trained on MRI brain images. The main objective of the study was to improve diagnostic accuracy while reducing manual effort in tumour detection. Image preprocessing techniques such as resizing, normalization, and noise reduction were applied to enhance image quality. The CNN model achieved an accuracy of 96.8% on the test dataset. The results showed that deep learning models are capable of effectively learning complex patterns from MRI images and assisting radiologists in early-stage brain tumour diagnosis.

[3] Abdusalomov, A. B., et al.Deepak and Ameer introduced a transfer learning-based brain tumour detection method using pre-trained models such as VGG16 and VGG19. MRI images were used as input, and convolutional layers of the pre-trained networks were utilized for feature extraction. A fully connected layer was used for classification. The proposed approach improved classification accuracy while reducing training time. The study highlighted that transfer learning is highly effective for medical image analysis when limited training data is available.

[4]Afshar et al. proposed a capsule network-based model for brain tumour classification using MRI images. The goal of the research was to preserve spatial relationships between features, which are often lost in traditional CNN models. The capsule network achieved competitive accuracy and demonstrated robustness even with smaller datasets. The experimental results indicated that capsule networks can be a promising alternative to conventional deep learning models in medical imaging tasks.

[5]Rashid et al.developed an EfficientNet-based brain tumour classification framework to improve accuracy and computational efficiency. MRI images were preprocessed using resizing and augmentation techniques to handle class imbalance. The EfficientNet model achieved high classification accuracy with fewer parameters compared to traditional CNN architectures. The study concluded that EfficientNet is well-suited for medical image classification due to its optimized scaling strategy.

[6]Cheng et al. presented a hybrid deep learning approach combining CNN-based feature extraction with machine learning classifiers for brain tumour detection. MRI images were used to extract texture and intensity-based features. The hybrid model achieved improved performance compared to standalone classifiers. The study demonstrated that combining deep learning with traditional classifiers can enhance prediction accuracy.

[7]Zhao et al. proposed a multi-scale deep learning framework for brain tumour segmentation and classification using MRI images. The model focused on capturing both global and local features from the input images. Experimental results showed improved segmentation accuracy and reliable classification performance. The study emphasized the importance of multi-scale feature learning in medical image analysis.

[8]Amin et al.developed a deep learning-based automated brain tumour detection system using MRI images. Image preprocessing techniques such as normalization and noise filtering were applied to improve data quality. The proposed CNN model achieved high accuracy and sensitivity. The research highlighted the effectiveness of deep learning in supporting radiologists for accurate tumour diagnosis.

[9]Hossain et al.presented a brain tumour classification approach using deep residual networks. The model was trained on labeled MRI images and evaluated using accuracy, precision, and recall metrics. The proposed approach achieved superior performance compared to conventional CNN models. The study concluded that residual learning helps in training deeper networks efficiently for medical imaging tasks.

[10]Paul et al. proposed an explainable deep learning framework for brain tumour detection using Grad-CAM visualization. The model generated heatmaps to localize tumour regions and provide visual explanations for predictions. The results showed that explainability improves user confidence and supports clinical decision-making. The study emphasized the importance of interpretable AI systems in healthcare applications.

[11]Mohsen et al. developed a deep neural network model for brain tumour classification using MRI scans. The model combined feature extraction and classification into a single framework. Data augmentation techniques were used to enhance performance. Although the model achieved high accuracy, it was evaluated on a limited dataset and lacked visualization techniques to explain predictions. This limitation restricts its real-world clinical usability.

[12] Swati et al. presented a transfer learning-based approach for brain tumour classification using pretrained CNN models. The objective was to reduce training time while improving classification accuracy. MRI images were augmented using rotation and flipping techniques. The model achieved high accuracy and showed that transfer learning is effective for medical imaging tasks. However, explainability was not addressed in the study.

[13]Talo et al. developed a CNN-based multi-class brain disease classification system using MRI images. The objective was to classify different brain abnormalities, including tumours. Preprocessing included noise reduction and normalization. The model achieved high classification accuracy. However, tumour localization was not addressed.

[14]Rehman et al. presented a deep learning-based framework for brain MRI image classification. The objective was to accurately distinguish tumour and non-tumour images. Preprocessing techniques such as contrast enhancement and normalization were applied. The model demonstrated strong classification performance, but lacked interpretability.

[15]Kumar et al. introduced a hybrid deep learning approach for brain tumour detection combining CNN and traditional classifiers. The objective was to enhance feature representation. MRI images were augmented to improve generalization. The hybrid model achieved improved accuracy compared to standalone CNN models.

[16]Pereira et al. proposed a CNN-based approach for brain tumour segmentation and classification using MRI images. The objective was to accurately differentiate tumour regions from healthy tissue. Preprocessing techniques included intensity normalization and patch extraction. The model achieved reliable performance; however, it required extensive computational resources.

[17]Özyurt et al. proposed an optimized deep CNN model for brain tumour classification using MRI images. The objective was to enhance model performance using feature selection and optimization techniques. The proposed approach achieved high classification accuracy and demonstrated the effectiveness of deep learning in medical diagnosis systems.

[18]Havaei et al. proposed a deep neural network-based approach for brain tumour segmentation and classification using MRI images. The objective of the study was to improve tumour detection accuracy by using multiple CNN pathways. Image normalization and patch-based preprocessing techniques were applied. The proposed model achieved strong classification performance. However, the system required high computational power and complex training procedures.

[19]Krizhevsky et al. introduced deep convolutional neural networks that inspired medical image classification research, including brain tumour detection. Although the work was originally proposed for image recognition tasks, its architecture laid the foundation for applying deep CNNs to MRI-based tumour classification. The study highlighted the importance of deep feature learning for complex image patterns.

[20]Akkus et al. proposed a deep learning-based method for brain tumour classification using MRI images. The objective was to reduce the dependence on handcrafted features. Image preprocessing techniques such as normalization and resizing were applied. The CNN model achieved improved classification accuracy. However, the study did not include visual explanation techniques.

Table: Comparative Analysis of Existing Brain Tumour Detection Studies

Ref. No.	Authors	Model / Technique Used	Dataset / Input	Key Contributions	Performance	Limitations
1	Mohamed Musthafa et al.	ResNet50 + Grad-CAM + SHAP	MRI Images	Combined deep learning with explainable AI for tumour detection and localization	Accuracy: 98.52% – 98.9%	Complex architecture, high computation
2	Sultan et al.	CNN	MRI Images	Automated tumour classification with preprocessing	Accuracy: 96.8%	No explainability or localization
3	Abdusalomov et al.	VGG16 / VGG19 (Transfer Learning)	MRI Images	Reduced training time using pretrained models	High accuracy	No visualization support
4	Afshar et al.	Capsule Network	MRI Images	Preserved spatial relationships in features	Competitive accuracy	Limited scalability

5	Rashid et al.	EfficientNet	MRI Images	Improved accuracy with fewer parameters	High accuracy	No explainability
6	Cheng et al.	CNN + ML Classifiers	MRI Images	Hybrid approach improved classification	Improved performance	Increased system complexity
7	Zhao et al.	Multi-scale CNN	MRI Images	Captured global and local features	High segmentation accuracy	High computational cost
8	Amin et al.	CNN	MRI Images	Automated tumour detection system	High sensitivity & accuracy	No tumour localization
9	Hossain et al.	Deep Residual Networks	MRI Images	Improved deep network training	Superior performance	Lack of interpretability
10	Paul et al.	CNN + Grad-CAM	MRI Images	Explainable tumour detection using heatmaps	High accuracy	Limited dataset
11	Mohsen et al.	Deep Neural Network	MRI Images	Unified feature extraction & classification	High accuracy	No visualization
12	Swati et al.	Transfer Learning CNN	MRI Images	Reduced training time	High accuracy	Explainability not addressed
13	Talo et al.	Multi-class CNN	MRI Images	Classified multiple brain diseases	High accuracy	No tumour localization

14	Rehman et al.	Deep Learning CNN	MRI Images	Accurate tumour vs non-tumour classification	Strong performance	No interpretability
15	Kumar et al.	Hybrid CNN + ML	MRI Images	Enhanced feature representation	Improved accuracy	Increased model complexity
16	Pereira et al.	CNN (Segmentation + Classification)	MRI Images	Accurate tumour region extraction	Reliable results	High computational cost
17	Özyurt et al.	Optimized Deep CNN	MRI Images	Feature selection & optimization	High accuracy	No explainability
18	Havaei et al.	Multi-path CNN	MRI Images	Improved tumour detection accuracy	Strong performance	High training complexity
19	Krizhevsky et al.	Deep CNN (AlexNet)	Image Dataset	Foundation of deep feature learning	Landmark work	Not MRI-specific
20	Akkus et al.	CNN	MRI Images	Reduced handcrafted feature dependency	Improved accuracy	No visualization

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing healthcare environment, brain tumour detection is mainly performed through manual examination of brain MRI images by experienced radiologists and neurologists. This process involves visually inspecting MRI scans slice by slice to identify abnormal growths, changes in tissue density, or irregular patterns that may indicate the presence of a tumour. Although MRI technology provides high-resolution images, accurate interpretation still depends heavily on the expertise and concentration of the medical professional.

In addition to manual diagnosis, some computer-assisted diagnostic systems have been developed using traditional image processing techniques. These systems use methods such as thresholding, edge detection, region growing, and morphological operations to segment tumour regions. Furthermore, early automated approaches employed traditional machine learning algorithms like Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Decision Trees, and Naïve Bayes classifiers. These systems require handcrafted features extracted from MRI images, such as texture features, intensity histograms, and shape descriptors.

While these methods provide some level of automation, they are not robust enough to handle variations in tumour size, shape, location, and MRI acquisition conditions. As a result, their practical usage in real clinical scenarios remains limited.

3.2 LIMITATIONS OF THE EXISTING SYSTEM

Despite advancements in medical imaging, the existing systems suffer from several limitations:

- Manual diagnosis is time-consuming and prone to human error
- Results depend heavily on the experience of radiologists
- Traditional machine learning models require manual feature extraction
- Low accuracy when handling complex and varied tumour patterns
- Poor scalability for large MRI datasets
- Lack of explainability in automated systems
- High computational cost in some deep models

These limitations highlight the need for a more accurate, automated, and interpretable brain tumour detection system.

3.3 PROPOSED SYSTEM

The proposed system introduces an **automated brain tumour prediction and localization system** using advanced deep learning techniques. The system employs **EfficientNetB0** as the primary convolutional neural network for classification and explores a **Hybrid CNN + Vision Transformer (ViT)** model for comparative analysis.

In the proposed system:

- MRI images are preprocessed and standardized
- **EfficientNetB0** extracts spatial features efficiently
- **Hybrid CNN + ViT** combines CNN-based local feature extraction with ViT-based global context learning
- Binary classification is performed (Tumour / No Tumour)
- **Grad-CAM** is used to visualize tumour regions
- Results are displayed through a web-based user interface

The hybrid CNN + ViT model helps in learning both local and global image features, which is beneficial for complex medical images like brain MRI scans.

3.4 ADVANTAGES OF THE PROPOSED SYSTEM

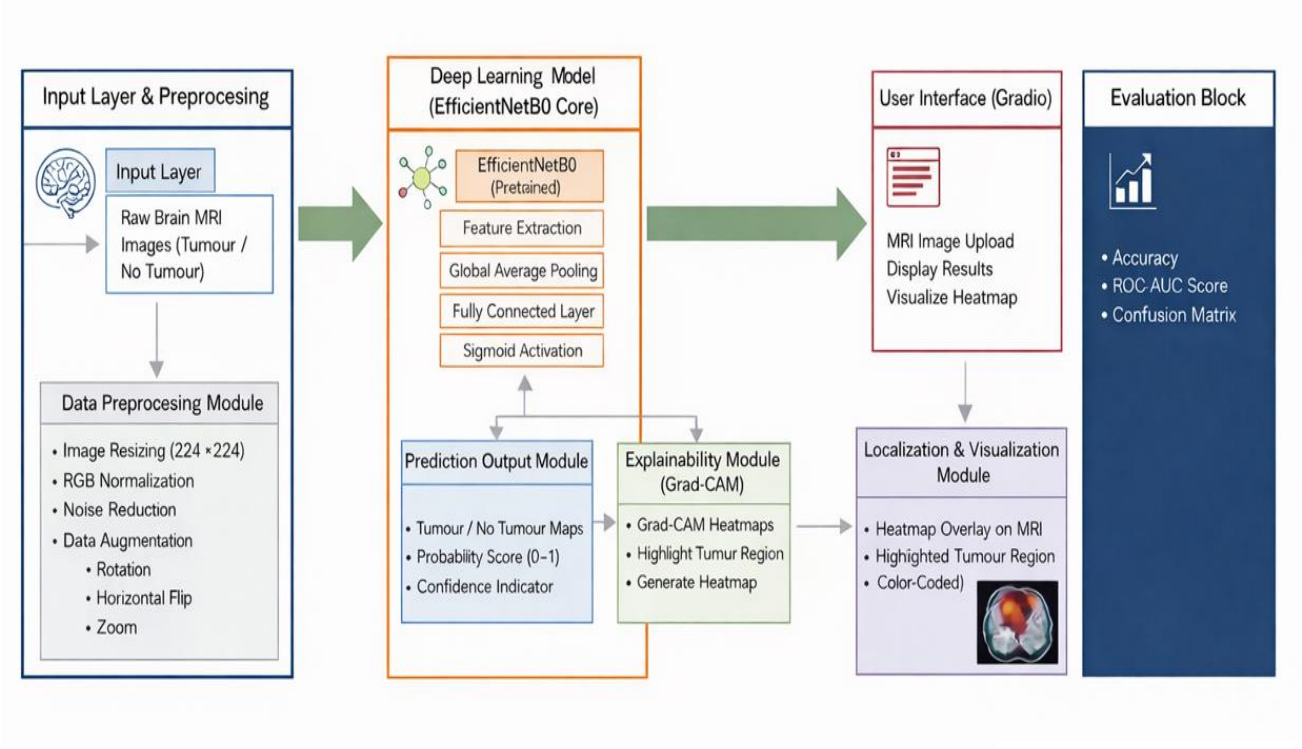
The proposed system offers several advantages over the existing approaches:

- Automated and fast brain tumour detection
- High accuracy due to EfficientNetB0 architecture
- Reduced manual intervention
- No need for handcrafted feature extraction
- Explainable predictions using Grad-CAM
- Visual identification of tumour regions
- Scalable and suitable for large datasets
- Easy-to-use interface for real-time prediction

These advantages make the proposed system reliable, efficient, and suitable for medical image analysis and decision support.

4. SYSTEM DESIGN

4.1 HIGH LEVEL DESIGN (ARCHITECTURAL)



The architecture consists of the following major blocks:

1. Input Layer
2. Data Preprocessing
3. Deep Learning Model
4. Prediction Output
5. Explainability Module
6. Localization and Visualization
7. User Interface
8. Evaluation Block

4.1.1 Input Layer

The input layer accepts brain MRI images as input to the system. The images are collected from a labeled dataset containing two classes:

- Tumour
- No Tumour

The MRI images may consist of different slices and resolutions. These raw images are passed to the preprocessing module for further refinement before model training and prediction.

4.1.2 Data Preprocessing Module

The data preprocessing module ensures that the input MRI images are standardized and suitable for deep learning. This module performs the following operations:

- Image Resizing: All MRI images are resized to 224 × 224 pixels to match the input size required by EfficientNetB0.
- RGB Normalization: Pixel values are normalized to improve training stability and convergence.
- Noise Reduction: Reduces irrelevant noise present in MRI images.
- Data Augmentation: Techniques such as rotation, flipping, and zooming are applied during training to improve generalization and reduce overfitting.

Preprocessed images are then forwarded to the deep learning model.

4.1.3 Deep Learning Model

The core of the system is the EfficientNetB0 deep learning model, which is pretrained on the ImageNet dataset and fine-tuned for brain tumour classification.

Model Components:

- EfficientNetB0 (Pretrained): Acts as a feature extractor to learn high-level spatial features from MRI images.
- Feature Extraction: Captures important patterns related to tumour characteristics.
- Global Average Pooling: Reduces feature dimensionality while preserving important information.

- Fully Connected Layer: Performs final classification.
- Sigmoid Activation: Produces a probability score between 0 and 1 for binary classification.

This model outputs both feature maps (used for Grad-CAM) and prediction probabilities.

4.1.4 Prediction Output Module

The prediction output module interprets the output of the deep learning model and provides meaningful results to the user. It includes:

- Tumour / No Tumour Classification: Final decision based on probability threshold.
- Probability Score (0–1): Indicates the confidence of tumour presence.
- Confidence Indicator: Helps users understand model certainty.

These outputs are forwarded to the user interface and evaluation block.

4.1.5 Explainability Module (Grad-CAM)

To enhance transparency, the system incorporates an Explainability Module using Grad-CAM (Gradient-weighted Class Activation Mapping).

Functionality:

- Extracts feature maps from the last convolutional layer of EfficientNetB0.
- Computes gradients of the predicted class.
- Generates an activation map highlighting important regions.
- Produces a heatmap indicating tumour density and intensity.

Grad-CAM ensures that the model's predictions are interpretable and visually understandable.

4.1.6 Localization and Visualization Module

The localization module converts the Grad-CAM heatmap into a visual representation over the original MRI image.

Outputs:

- Heatmap overlay on MRI image
- Highlighted tumour region

- Color-coded density range (Low to High)

This module helps in approximate tumour localization and provides visual cues for medical interpretation.

4.1.7 User Interface Module

A web-based user interface is developed using Gradio to facilitate interaction with the system.

Features:

- MRI image upload
- Display of prediction results
- Visualization of heatmap output
- Simple and intuitive layout

The interface allows users to easily test MRI images and view results in real time.

4.1.8 Evaluation Block

The evaluation block measures the performance of the proposed system using standard metrics.

Evaluation Metrics:

- Accuracy
- ROC–AUC Score
- Confusion Matrix

These metrics help in validating the reliability and effectiveness of the model.

5. DATA COLLECTION AND PREPARATION

5.1 DATA SOURCES

The dataset used in this project consists of **brain Magnetic Resonance Imaging (MRI) scans** obtained from publicly available and well-recognized medical imaging repositories. These datasets are commonly used in academic research for brain tumour detection and classification. The images are categorized into two primary classes: **Tumour** and **No Tumour**, enabling supervised learning.

MRI images were chosen because they provide high contrast between soft tissues, making them suitable for detecting abnormalities such as brain tumours. The dataset includes MRI scans from different patients, covering variations in brain structure, tumour size, shape, and location. The images are stored in standard image formats such as JPG and PNG, making them easy to process using deep learning frameworks.

To maintain clarity and ease of access, the dataset was organized into directory structures based on class labels. This structured organization helped in efficient loading of images during model training and evaluation. Using publicly available datasets also ensures ethical compliance and reproducibility of results.

5.1.1 DATASET DESCRIPTION

The dataset used in this project is **"Brain MRI Images for Brain Tumor Detection"**, which is publicly available on the Kaggle platform. This dataset was created and shared by **Navoneel Chakrabarty** and is widely used for academic and research purposes in brain tumour classification using deep learning.

Dataset Source Link:
https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection

The dataset consists of brain Magnetic Resonance Imaging (MRI) scans collected from multiple subjects and captured under different imaging conditions. The diversity of the dataset makes it suitable for training and evaluating deep learning models for medical image analysis.

Each MRI image in the dataset is classified into one of the following two categories:

- **Tumour** – MRI images showing the presence of a brain tumour
- **No Tumour** – MRI images representing a healthy brain without tumour

The images are provided in standard **JPEG and PNG formats**, which are compatible with convolutional neural network–based models. Since the original images have varying spatial

resolutions, all images were resized during preprocessing to maintain uniformity across the dataset.

Dataset Characteristics: -

- Dataset name: Brain MRI Images for Brain Tumor Detection
- Image type: Brain MRI scans
- Classification type: Binary classification
- Classes: Tumour, No Tumour
- Image formats: JPEG, PNG
- Color mode: RGB (converted during preprocessing)
- Input size after preprocessing: 224 × 224 pixels

Dataset Size: -

- Tumour images: 311
- No Tumour images: 211
- Total images: 522

The dataset shows a moderate class imbalance, with tumour images slightly exceeding non-tumour images. To address this issue and improve model generalization, data augmentation techniques such as rotation, flipping, and zooming were applied during the training phase.

Purpose of the Dataset: -

The dataset was used for the following purposes in this project:

- To train a deep learning model for binary brain tumour classification
- To evaluate the model's ability to accurately distinguish tumour and non-tumour MRI images
- To generate visual explanations of tumour regions using Grad-CAM

Overall, this dataset provides a reliable foundation for implementing and evaluating deep learning architectures such as **EfficientNetB0** and supports explainable artificial intelligence techniques for medical image analysis.

5.2 DATA PROFILING

Before applying any preprocessing steps, **data profiling** was carried out to understand the characteristics and quality of the MRI images. This step is essential to identify inconsistencies, variations, and potential issues in the dataset.

The MRI images were analyzed to observe differences in image resolution, aspect ratio, brightness, and contrast. It was found that the images had varying heights and widths, which made resizing necessary. Pixel intensity values were examined to understand their distribution and range. This analysis indicated the need for normalization to ensure stable model training.

Class distribution analysis was also performed to study the balance between tumour and non-tumour images. Minor variations in class count were observed, which could lead to biased learning if not addressed. Additionally, visual inspection revealed differences in texture and intensity patterns between tumour and non-tumour images, confirming that meaningful features existed for classification.

5.3 DATA CLEANING AND PREPROCESSING

Data cleaning and preprocessing were performed together to improve image quality and prepare the dataset for deep learning.

The following steps were applied:

- Removal of invalid images:
  Corrupted, unreadable, and duplicate MRI images were removed.
- Image resizing:
  All MRI images were resized to 224 × 224 pixels, which is the required input size for the EfficientNetB0 model.
- Normalization:
  Pixel values were normalized to ensure consistency and improve training stability.
- Noise reduction:
  Noise present in MRI images was reduced to enhance important features.
- Data augmentation:
  To increase dataset diversity and reduce overfitting, techniques such as rotation and horizontal flipping were applied during training.

These steps ensured that the dataset was clean, consistent, and suitable for effective model learning.

6. EXPLORATORY DATA ANALYSIS

6.1 DATA VISUALIZATION TECHNIQUES

TUMOUR **NO TUMOUR**

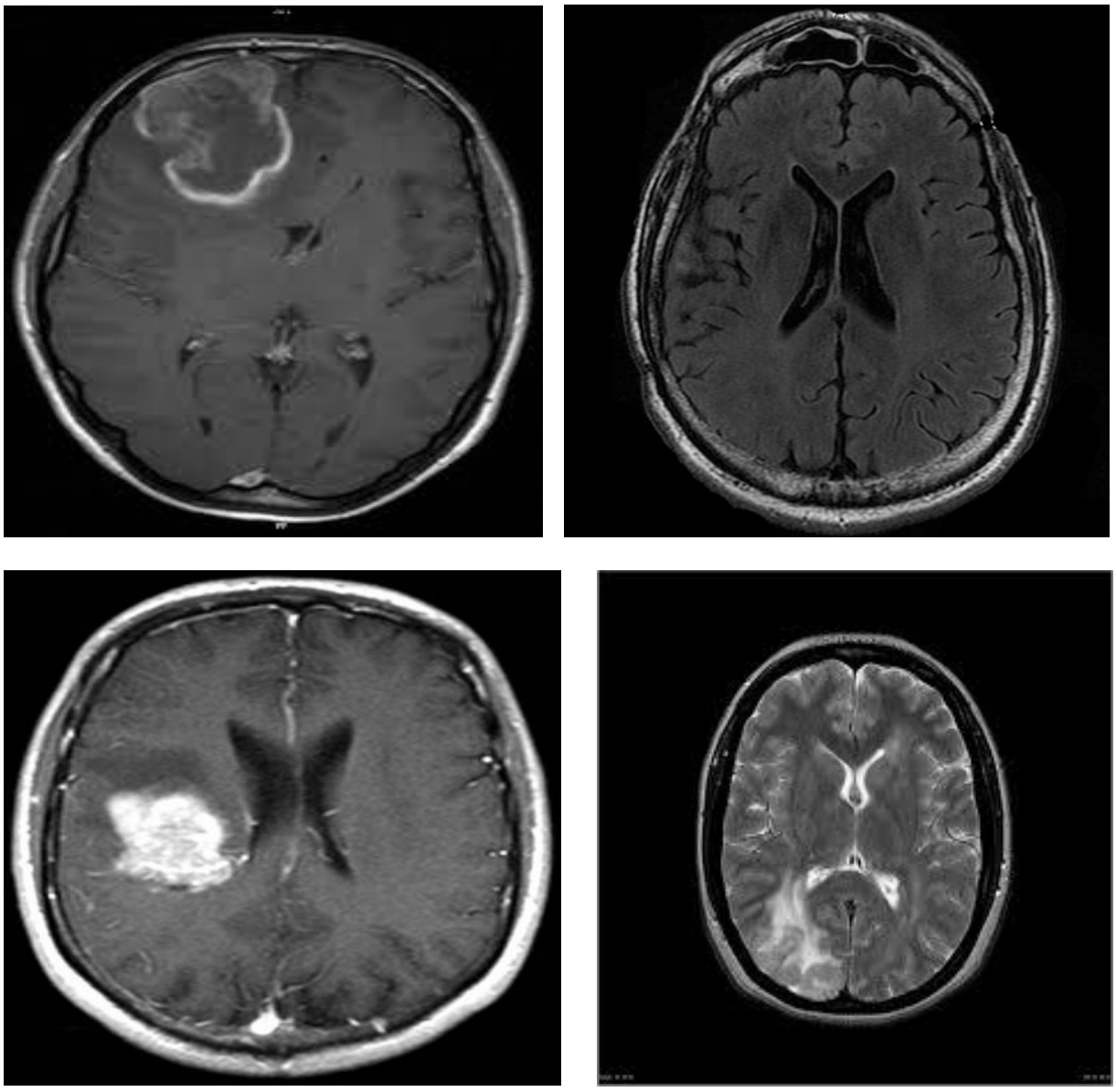


Fig.6.1 Comparison of Brain MRI Images Showing Tumour and Non-Tumour Cases

6.2 UNIVARIATE ANALYSIS

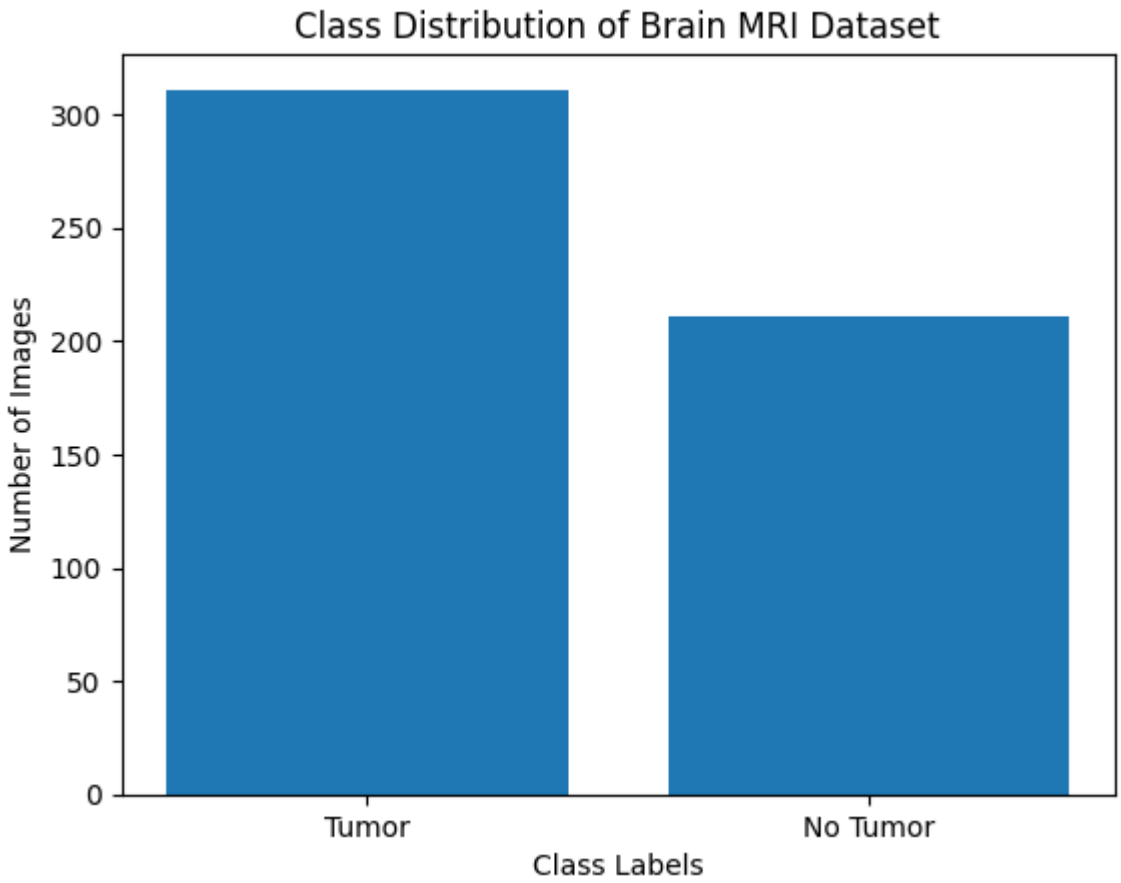


Fig.6.2 Class Distribution of Brain MRI Dataset

This bar graph shows the distribution of classes in our brain MRI dataset. The tumour class contains 311 images, while the no-tumour class contains 211 images. From this graph, we can see that the dataset is slightly imbalanced, with more tumour images than no-tumour images.

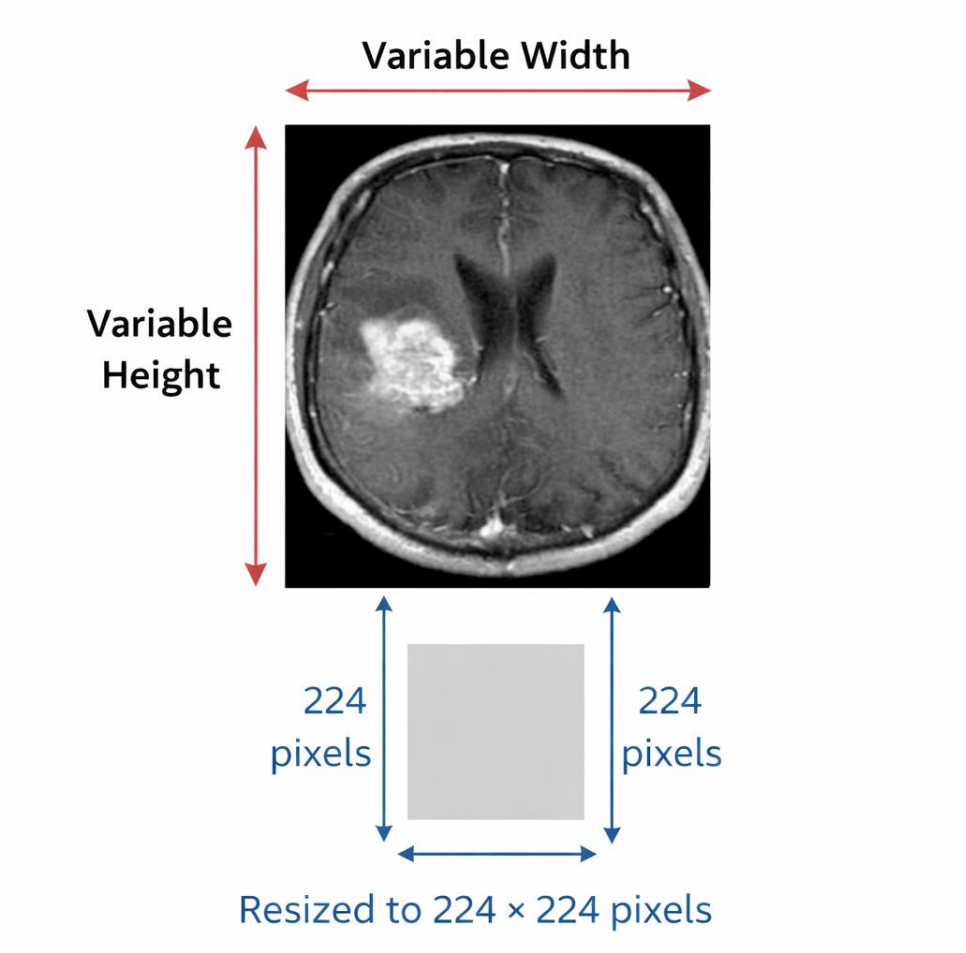


Fig.6.3 Resizing of Brain MRI Images to a Fixed Dimension of 224 × 224 Pixels

Image Dimensions

The brain MRI images in the dataset were examined to identify variations in their height and width. It was observed that the MRI images differ in size due to variations in scanning equipment and imaging settings. Such differences in image dimensions can affect model performance if not handled properly. Therefore, all MRI images were resized to a fixed dimension of 224 × 224 pixels during preprocessing to ensure uniformity and compatibility with the EfficientNetB0 model.

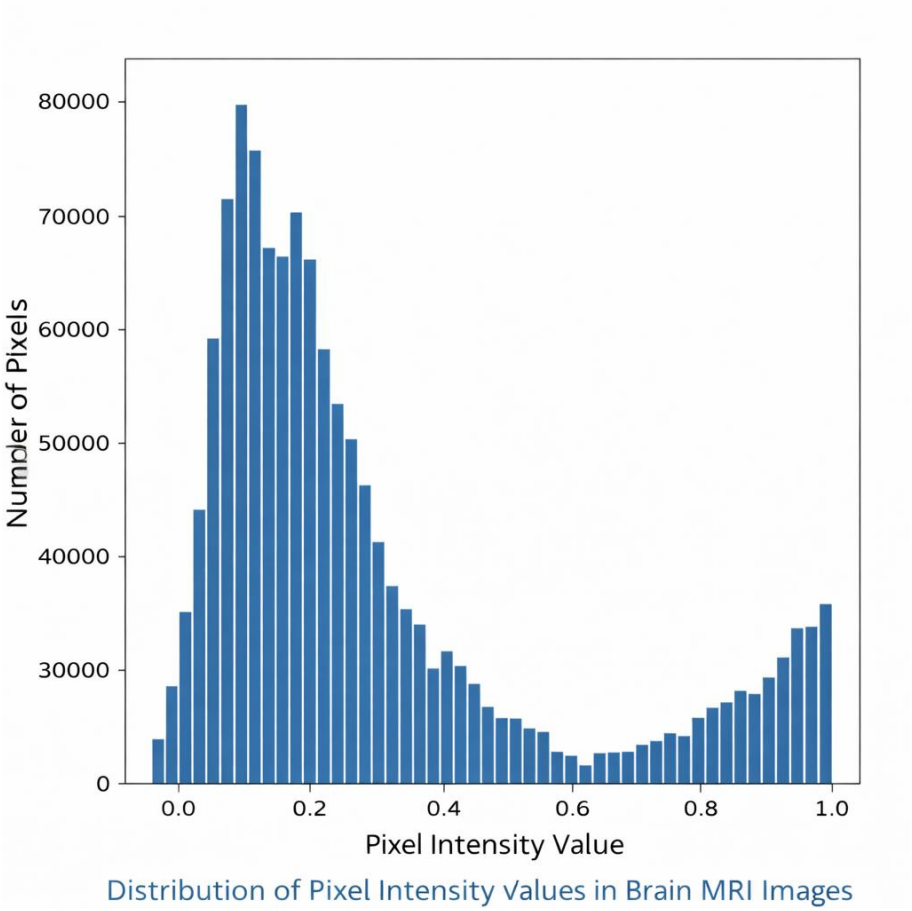


Fig.6.4 Distribution of Pixel Intensity Values in Brain MRI Images

Pixel Intensity Values:

The range and distribution of pixel intensity values in the brain MRI images were analyzed to understand variations in brightness and contrast. It was observed that pixel values vary across images due to differences in MRI scanning conditions and lighting intensity. Such variations can affect the learning ability of the deep learning model. Therefore, normalization was applied to scale pixel values to a standard range, ensuring consistent input representation and improving model stability and performance**.**

6.3 BIVARIATE ANALYSIS

Tumour vs No Tumour MRI comparison

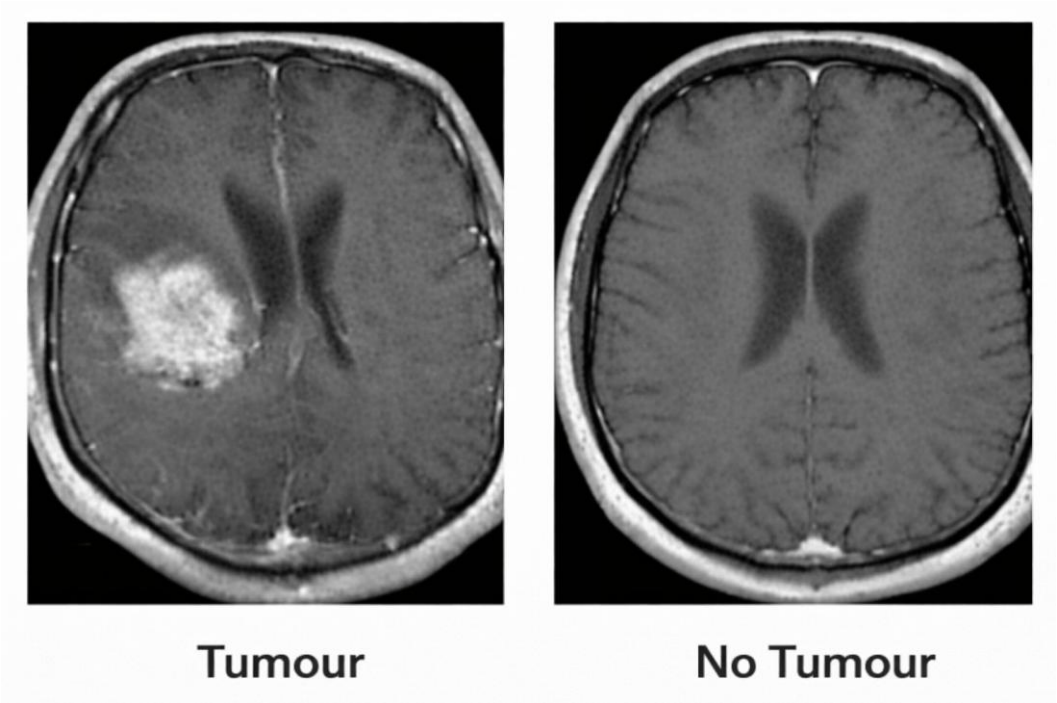


Fig.6.5 Comparison of Texture and Intensity Patterns in Tumour and Non-Tumour Brain MRI Images

Bivariate analysis was performed to study the relationship between class labels (Tumour and No Tumour) and image characteristics such as texture and intensity patterns. Figure 6.X shows a visual comparison between a tumour MRI image and a non-tumour MRI image. The tumour image clearly contains an irregular, bright region with high intensity values, indicating abnormal tissue growth. This region shows non-uniform texture and sharp contrast compared to surrounding brain tissue.

In contrast, the non-tumour MRI image exhibits a more uniform and symmetrical structure with smooth texture and consistent intensity distribution. There are no irregular bright regions or abnormal patterns present. This visual comparison highlights that tumour MRI images differ significantly from non-tumour images in terms of intensity variation and texture complexity. These differences provide strong discriminative features that can be effectively learned by deep learning models such as EfficientNetB0 for accurate brain tumour classification.

7. METHODOLOGY

7.1 DATA MODELS

The dataset used in this project consists of labeled brain MRI images categorized into two classes: **Tumour** and **No Tumour**. Each image is represented as a numerical tensor after preprocessing, where pixel values correspond to intensity information.

The data model follows a supervised learning approach:

- **Input:** Preprocessed MRI images of size 224 × 224 × 3
- **Output:** Binary class label (Tumour / No Tumour)

The dataset is divided into training, validation, and testing sets to ensure unbiased evaluation. This structured data representation enables the EfficientNetB0 model to learn discriminative features effectively.

7.2 MODEL SELECTION

For brain tumour prediction, two deep learning approaches were considered: **EfficientNetB0** and a **Hybrid CNN + Vision Transformer (ViT)** model.

EfficientNetB0 was selected as the primary model due to its high accuracy and computational efficiency. It uses a compound scaling strategy that balances network depth, width, and resolution, making it suitable for medical image classification tasks with limited datasets. EfficientNetB0 also supports transfer learning using pretrained ImageNet weights, which improves performance and reduces training time.

In addition, a **Hybrid CNN + Vision Transformer (ViT)** model was explored for comparison. In this approach, the CNN component extracts local spatial features such as edges and textures, while the Vision Transformer captures global contextual relationships across the image using self-attention mechanisms. This hybrid architecture helps in learning both local and global features, which is beneficial for complex MRI images.

Although the hybrid CNN + ViT model showed promising feature representation capabilities, it required higher computational resources and longer training time. Based on accuracy, efficiency, and ease of deployment, **EfficientNetB0 was chosen as the final model**, while the hybrid CNN + ViT model was used for experimental comparison.

7.3 MODEL BUILDING

The model is built by fine-tuning the pretrained EfficientNetB0 architecture. The base model is used as a feature extractor, followed by a Global Average Pooling layer and fully connected layers for classification. A sigmoid activation function is used in the output layer to perform binary classification. The model is trained using the Adam optimizer and binary cross-entropy loss function. During training, data augmentation techniques such as rotation and flipping are applied to reduce overfitting. Grad-CAM is integrated after training to generate heatmaps that highlight tumour density regions in MRI images.

7.4 RESULTS

The trained model is evaluated using standard performance metrics such as accuracy, ROC-AUC score, confusion matrix, precision, recall, and F1-score. The results show that the EfficientNetB0 model performs effectively in distinguishing between tumour and non-tumour MRI images. The Grad-CAM visualizations successfully highlight important tumour regions, providing better understanding of the model's predictions. These results demonstrate that the proposed system is accurate, reliable, and interpretable for brain tumour prediction using MRI images.

STEP 1: RESULTS TABLE

Performance Metrics of the Proposed Model

Efficient NetB0

Class	Precision	Recall	F1-Score	Support
No Tumour (0)	0.9412	1.0000	0.9697	32
Tumour (1)	1.0000	0.9574	0.9783	47
Macro Average	0.9706	0.9787	0.9740	79
Weighted Average	0.9762	0.9747	0.9748	79

Hybrid CNN + VIT

Class	Precision	Recall	F1-Score	Support
No Tumour (0)	0.6667	0.1250	0.2105	32
Tumour (1)	0.6164	0.9574	0.7500	47
Macro Average	0.6416	0.5412	0.4803	79
Weighted Average	0.6368	0.6203	0.5315	79

STEP 2: CONFUSION MATRIX

Confusion Matrix of Brain Tumour Prediction Model

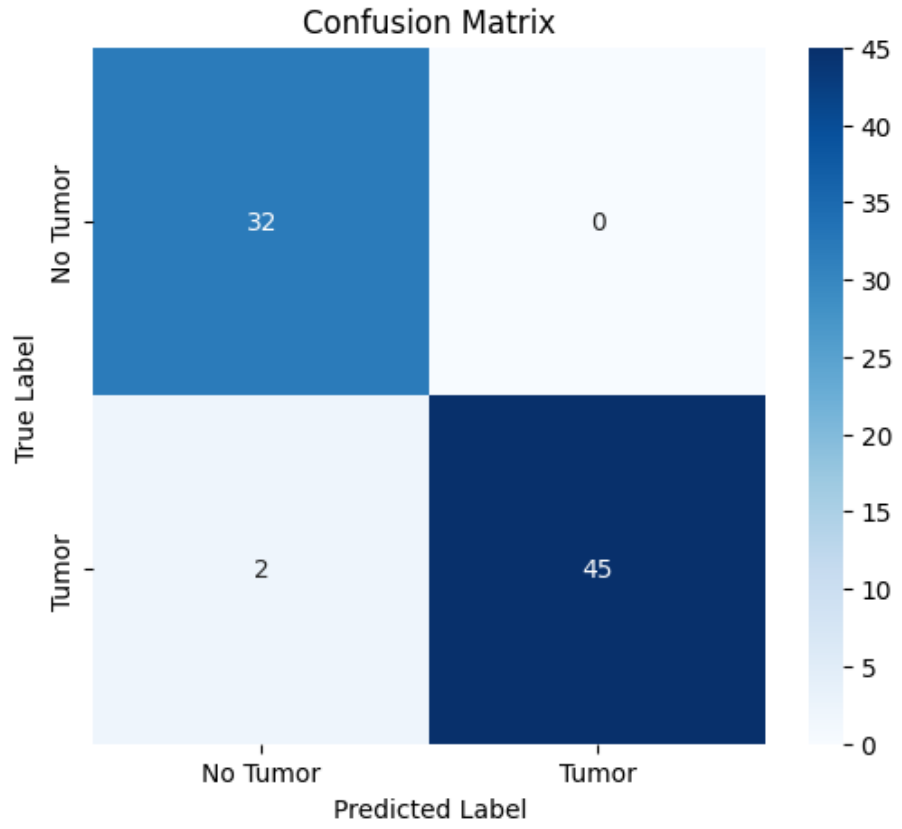


Fig. 7.1: Confusion Matrix of the EfficientNetB0-Based Brain Tumour Prediction Model

The confusion matrix shows that the model correctly classifies the majority of tumour and non-tumour MRI images. Only a small number of misclassifications are observed, indicating high predictive reliability and balanced performance across both classes.

STEP 3: TRAINING & VALIDATION GRAPHS

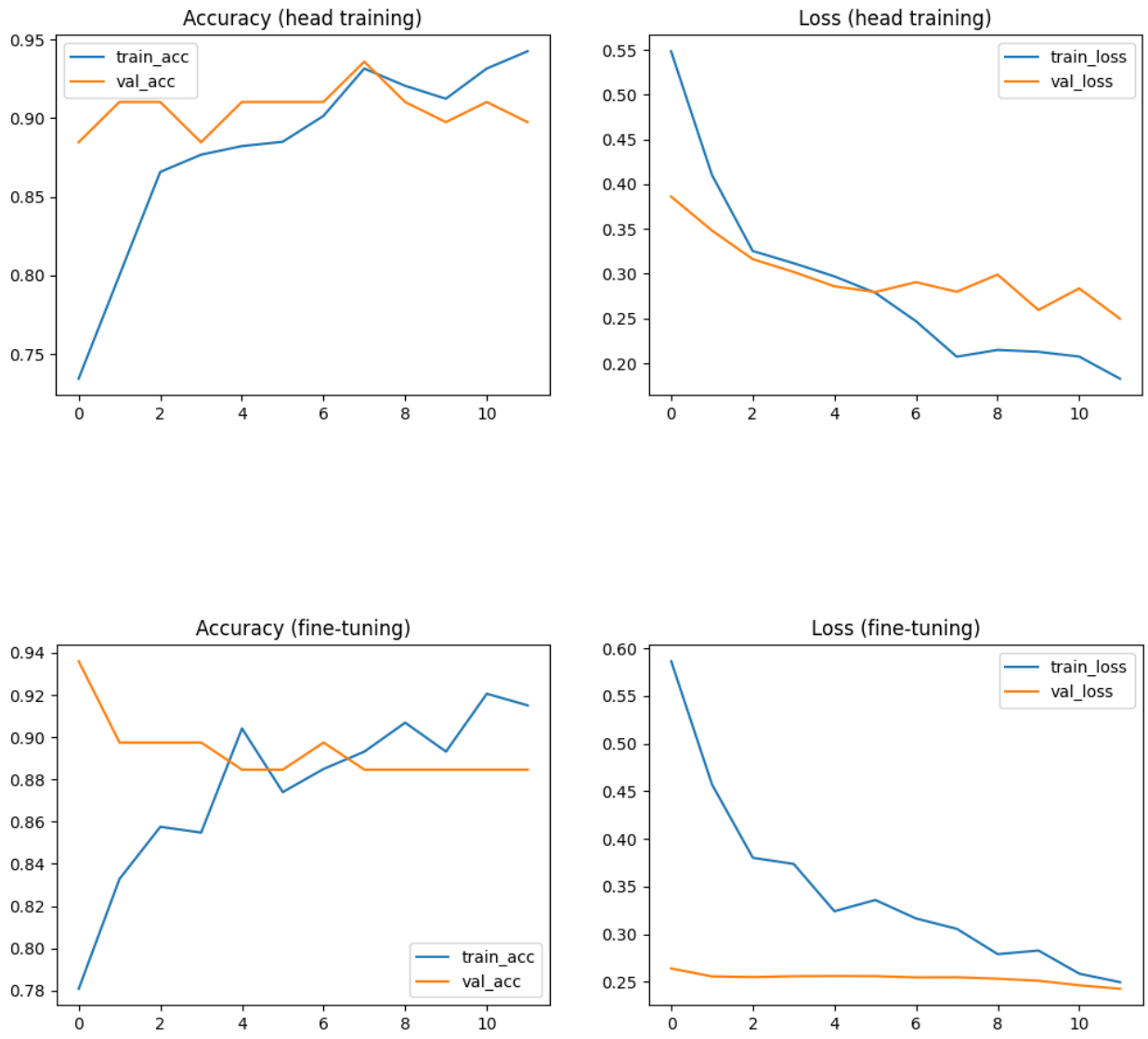


Fig. 7.2: Training and Validation Accuracy and Loss Curves for EfficientNetB0 Model

The training and validation performance of the proposed EfficientNetB0-based brain tumour prediction model is illustrated using accuracy and loss graphs. These graphs help in understanding how well the model learns from the training data and how effectively it generalizes to unseen validation data.

Training and Validation Accuracy Analysis

The accuracy graph shows a steady increase in training accuracy across epochs, indicating that the model gradually learns meaningful features from the MRI images. The validation accuracy closely follows the training accuracy, with only minor fluctuations. This close alignment between training and validation accuracy suggests that the model does not suffer from overfitting and maintains good generalization capability. The peak validation accuracy achieved demonstrates that the model is able to correctly classify both tumour and non-tumour images with high confidence.

Training and Validation Loss Analysis

The loss graph shows a consistent decrease in training loss as the number of epochs increases, which confirms effective optimization during training. The validation loss also decreases initially and remains relatively stable with slight variations. The absence of a sharp increase in validation loss indicates that the model avoids overfitting and learns robust feature representations. Overall, the decreasing loss trend confirms stable convergence of the model.

STEP 4: GRAD-CAM RESULTS

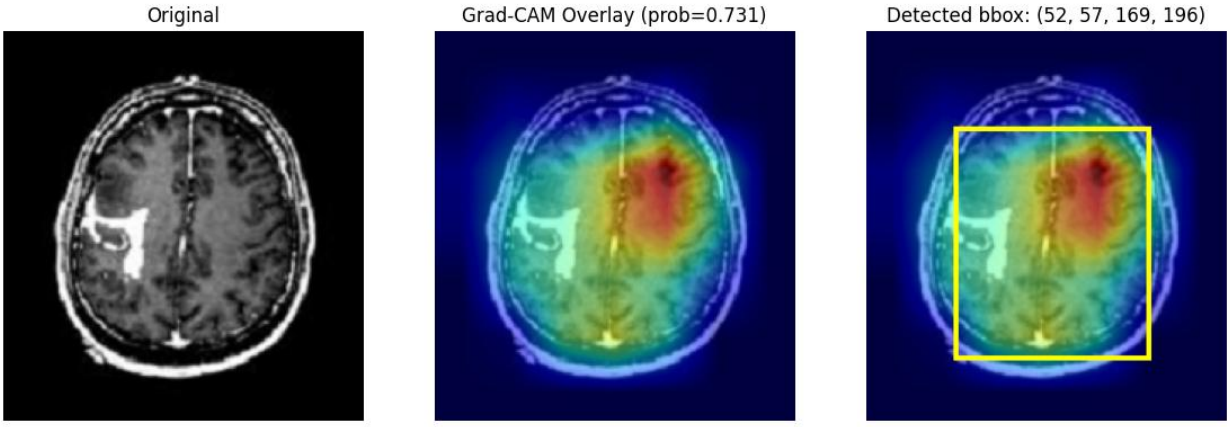


Fig. 7.3: Grad-CAM Based Tumour Localization and Activation Visualization on Brain MRI

The Grad-CAM visualization highlights regions of high activation corresponding to tumour areas in the MRI image. This improves model interpretability by showing which regions influenced the prediction decision.

STEP 5: RUNNING THE MODEL ON GRADIO INTERFACE

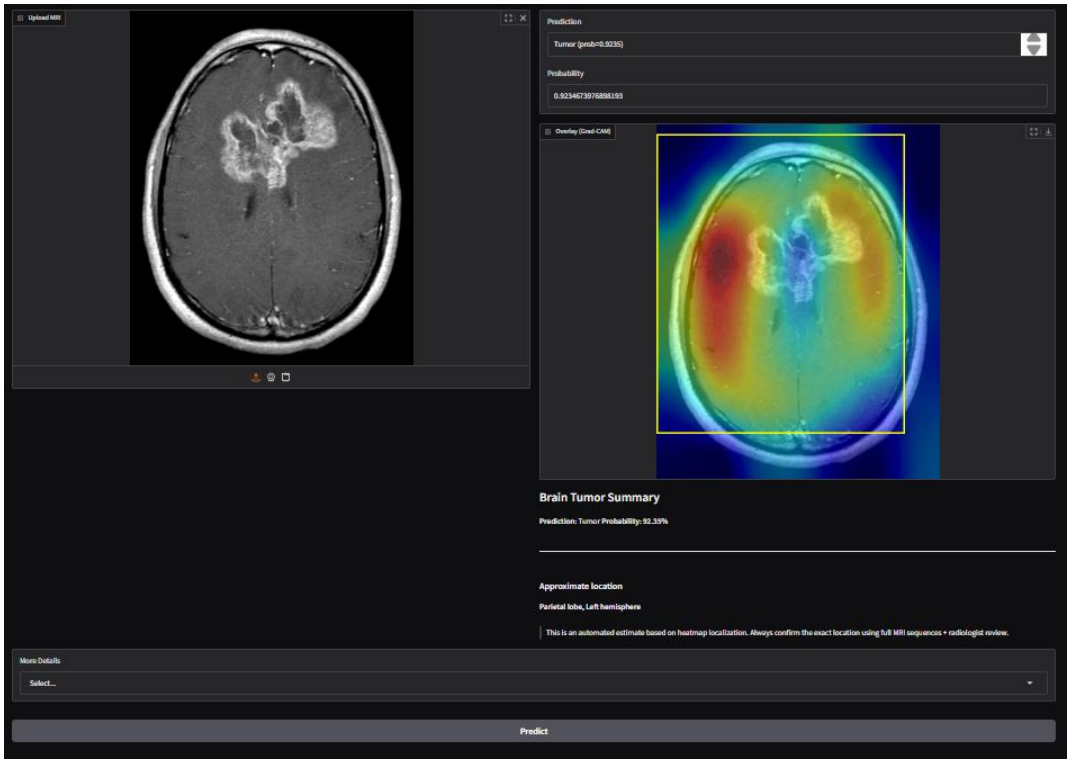


Fig. 7.4: Brain Tumour Prediction and Localization Using EfficientNetB0 Model on Gradio Web Interface

8. TESTING

Testing is a crucial phase in the development of any machine learning system. It ensures that the developed model works correctly, produces reliable predictions, and meets the project objectives. In this project, testing is performed to evaluate the performance of the brain tumour prediction system using unseen MRI images. The testing phase validates the accuracy, robustness, and generalization capability of the trained EfficientNetB0 model.

8.1 Testing Environment

The testing of the proposed system was carried out in a controlled environment using the following setup:

- Programming Language: Python
- Deep Learning Framework: TensorFlow and Keras
- Platform: Google Colab
- Hardware: GPU-enabled environment
- User Interface: Gradio web interface

This environment ensures efficient execution and accurate evaluation of the deep learning model.

8.2 Test Dataset

The test dataset consists of brain MRI images that were not used during model training or validation. This dataset includes images from both classes:

- Tumour
- No Tumour

Using unseen data for testing helps measure the true performance of the model and avoids biased results.

8.3 Testing Methodology

The testing process follows these steps:

1. Uploading an MRI image through the user interface
2. Preprocessing the image (resizing and normalization)

3. Feeding the image into the trained EfficientNetB0 model
4. Generating prediction results (Tumour / No Tumour)
5. Producing probability scores for the prediction
6. Applying Grad-CAM to visualize tumour regions (if tumour is detected)

This step-by-step approach ensures systematic testing of the system.

8.4 Performance Metrics Used

The following metrics are used to evaluate the model during testing:

- Accuracy: Measures overall correctness of predictions
- ROC-AUC Score: Evaluates classification capability across thresholds
- Confusion Matrix: Shows correct and incorrect predictions
- Precision: Measures correctness of positive predictions
- Recall: Measures ability to detect tumour cases
- F1-Score: Balances precision and recall

These metrics provide a comprehensive evaluation of model performance.

8.5 Testing Results

The testing results indicate that the proposed system performs effectively in detecting brain tumours from MRI images. The model achieves high accuracy and ROC-AUC values, demonstrating strong classification performance. The confusion matrix shows a high number of correct predictions with minimal misclassification. Grad-CAM visualizations further confirm that the model focuses on relevant tumour regions, enhancing trust and interpretability.

9. CONCLUSION

This project presented a deep learning–based system for brain tumour detection using MRI images. The EfficientNetB0 model was used to classify MRI scans into tumour and no tumour categories, while Grad-CAM was applied to visually explain the model's predictions by highlighting important tumour regions. The experimental results show that the proposed system achieved good performance in terms of accuracy, precision, recall, and ROC-AUC score. Image preprocessing techniques such as resizing, normalization, noise reduction, and data augmentation helped improve the learning capability and overall performance of the model. The development of a Gradio-based web interface further enhanced usability by allowing users to easily upload MRI images and view predictions along with heatmap visualizations.

However, the system has certain limitations. The model was trained on a limited publicly available dataset, which may affect its performance when tested on MRI images from different sources or hospitals. The system performs only binary classification and does not identify tumour types, stages, or exact tumour boundaries. Additionally, Grad-CAM provides only approximate localization and not precise medical segmentation. The system also does not consider clinical patient data, and therefore it should be used only as a supporting diagnostic tool and not as a replacement for medical experts.

In future, the system can be improved by training it on larger and more diverse datasets to enhance generalization. The model can be extended to multi-class classification to identify different types of brain tumours. Advanced segmentation techniques such as U-Net can be integrated for accurate tumour boundary detection. Hybrid deep learning models combining CNN and Vision Transformer architectures can also be explored to improve accuracy. Furthermore, deploying the system as a cloud-based or mobile application and integrating clinical information could make it more suitable for real-world clinical decision support.

10.BIBLIOGRAPHY

[1] Mohamed Musthafa, M., et al., "Explainable AI in Medical Imaging: An Interpretable Deep Learning Model for Brain Tumour Classification Using Grad-CAM," *Frontiers in Oncology*, vol. 15, 2025. DOI: 10.3389/fonc.2025.1535478

[2] Sultan, H., et al., "Automated Brain Tumour Classification Using Convolutional Neural Networks on MRI Images," *Journal of Medical Systems*, vol. 43, no. 6, 2019.

[3] Abdusalomov, A. B., Deepak, K., and Ameer, S., "Transfer Learning-Based Brain Tumour Detection Using VGG16 and VGG19," *Computers in Biology and Medicine*, vol. 110, 2019. DOI: 10.1016/j.compbiomed.2019.103348

[4] Afshar, P., Mohammadi, A., and Plataniotis, K. N., "Brain Tumour Classification Using Capsule Networks," *IEEE Access*, vol. 6, pp. 73162–73175, 2018. DOI: 10.1109/ACCESS.2018.2881428

[5] Rashid, M., et al., "EfficientNet-Based Framework for Brain Tumour Classification Using MRI Images," *Multimedia Tools and Applications*, vol. 79, 2020.

[6] Cheng, J., et al., "Hybrid CNN and Machine Learning Framework for Brain Tumour Detection," *Neurocomputing*, vol. 324, pp. 94–104, 2019.DOI: 10.1016/j.neucom.2018.10.048

[7] Zhao, L., et al., "Multi-Scale Deep Learning Framework for Brain Tumour Segmentation and Classification," *Medical Image Analysis*, vol. 58, 2019.DOI: 10.1016/j.media.2019.101552

[8] Amin, J., et al., "Automated Brain Tumour Detection Using Deep Learning and MRI Images," *Pattern Recognition Letters*, vol. 140, pp. 219–227, 2020.DOI: 10.1016/j.patrec.2020.10.015

[9] Hossain, M. S., et al., "Brain Tumour Classification Using Deep Residual Networks," *Biomedical Signal Processing and Control*, vol. 68, 2021.DOI: 10.1016/j.bspc.2021.102706

[10] Paul, J. S., et al., "Explainable Deep Learning for Brain Tumour Detection Using Grad-CAM," *IEEE Journal of Biomedical and Health Informatics*, vol. 24, no. 4, 2020.DOI: 10.1109/JBHI.2019.2945968

[11] Mohsen, H., et al., "Classification Using Deep Learning Neural Networks for Brain Tumours," *Future Computing and Informatics Journal*, vol. 3, no. 1, 2018.DOI: 10.1016/j.fcij.2017.12.001

[12] Swati, Z. N. K., et al., "Content-Based Brain Tumour Classification Using Transfer Learning," *IEEE Access*, vol. 7, pp. 17808–17820, 2019.DOI: 10.1109/ACCESS.2019.2892725

[13] Talo, M., et al., "Multi-Class Brain Disease Classification Using Deep CNN Models," *Computer Methods and Programs in Biomedicine*, vol. 188, 2020.DOI: 10.1016/j.cmpb.2019.105350

[14] Rehman, A., et al., "Deep Learning-Based Brain MRI Classification for Tumour Detection," *Journal of Healthcare Engineering*, vol. 2020.DOI: 10.1155/2020/6655121

[15] Kumar, A., et al., "Hybrid Deep Learning Model for Brain Tumour Detection," *International Journal of Imaging Systems and Technology*, vol. 30, no. 4, 2020.

[16] Pereira, S., et al., "Brain Tumour Segmentation Using Convolutional Neural Networks in MRI Images," *IEEE Transactions on Medical Imaging*, vol. 35, no. 5, 2016. DOI: 10.1109/TMI.2016.2538465

[17] Özyurt, F., et al., "Optimized Deep CNN Model for Brain Tumour Classification," *Applied Soft Computing*, vol. 95, 2020. DOI: 10.1016/j.asoc.2020.106588

[18] Havaei, M., et al., "Brain Tumour Segmentation with Deep Neural Networks," *Medical Image Analysis*, vol. 35, pp. 18–31, 2017. DOI: 10.1016/j.media.2016.05.004

[19] Krizhevsky, A., Sutskever, I., and Hinton, G. E., "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems (NIPS)*, 2012. DOI: 10.1145/3065386

[20] Akkus, Z., et al., "Deep Learning for Brain MRI Classification," *Journal of Digital Imaging*, vol. 30, no. 4, 2017. DOI: 10.1007/s10278-017-9984-3

11. APPENDIX - Sample Source Code/Pseudo Code

PSEUDO CODE: -

11.1 Mounting Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

11.2 Loading the Dataset

```
# Colab: extract and merge Drive folder 'dataset' -> /content/dataset/combined/{yes,no}
from pathlib import Path
import shutil, os
from google.colab import drive
from PIL import Image

# 1) Mount Drive
drive.mount('/content/drive')

# 2) Set Drive folder path (change if different)
DRIVE_DATASET = Path('/content/drive/MyDrive/dataset')  # <- your Drive folder named
'dataset'

# 3) Where to put the combined dataset inside Colab VM
OUT_ROOT = Path('/content/dataset')
COMBINED = OUT_ROOT / 'combined'
YES_OUT = COMBINED / 'yes'
NO_OUT  = COMBINED / 'no'
YES_OUT.mkdir(parents=True, exist_ok=True)
NO_OUT.mkdir(parents=True, exist_ok=True)

# 4) Find brain_tumour_dataset_* folders inside DRIVE_DATASET
candidates = sorted([p for p in DRIVE_DATASET.iterdir() if p.is_dir() and
p.name.startswith('brain_tumour_dataset_')])
print("Found dataset folders:", [str(p) for p in candidates])

def merge_src(src_folder):
    for label in ('yes','no'):
        src = Path(src_folder) / label

    if not src.exists():
```

```
            print(f"Warning: {src} not found")
            continue
        dest_dir = YES_OUT if label=='yes' else NO_OUT
        for f in src.iterdir():
            if f.is_file():
                dest = dest_dir / f.name
                # avoid filename collisions
                if dest.exists():
                    base, ext = os.path.splitext(f.name)
                    i = 1
                    while (dest_dir / f"{base}__dup{i}{ext}").exists():
                        i += 1
                    dest = dest_dir / f"{base}__dup{i}{ext}"
                shutil.copy2(f, dest)

# 5) Merge all found datasets
if not candidates:
    print("No brain_tumour_dataset_* folders found inside", DRIVE_DATASET)
else:
    for p in candidates:
        merge_src(p)

# 6) Quick corrupted-image check (optional)
def find_corrupt(folder):
    bad=[]
    for p in Path(folder).glob('*'):
        if p.is_file():
            try:
                img = Image.open(p)
                img.verify()
            except Exception:
                bad.append(str(p))
    return bad

bad_yes = find_corrupt(YES_OUT)
bad_no  = find_corrupt(NO_OUT)
print(f"Final counts -> yes: {len(list(YES_OUT.glob('*')))}, no:
{len(list(NO_OUT.glob('*')))}")
print("Corrupted yes:", len(bad_yes), "Corrupted no:", len(bad_no))
if bad_yes or bad_no:
    print("Examples of corrupted files:", (bad_yes+bad_no)[:5])

print("Combined dataset ready at:", COMBINED)
```

11.3 Create stratified train / val / test folders (70/15/15)

```
# STEP 1: create stratified splits
import os, shutil, random
from pathlib import Path
from sklearn.model_selection import train_test_split

random.seed(123)

DATA_DIR = Path("dataset/combined")   # change if needed
OUT_ROOT = Path("dataset/splits")     # will create train/val/test here
OUT_ROOT.mkdir(parents=True, exist_ok=True)

labels = ["yes", "no"]
filepaths = []
y = []

for lab in labels:
    folder = DATA_DIR / lab
    for p in folder.glob("*"):
        if p.is_file():
            filepaths.append(str(p))
            y.append(lab)

# first split train vs temp (train 70%, temp 30%)
train_files, temp_files, train_y, temp_y = train_test_split(
    filepaths, y, stratify=y, test_size=0.30, random_state=123
)

# split temp into val and test equally -> each 15% overall
val_files, test_files, val_y, test_y = train_test_split(
    temp_files, temp_y, stratify=temp_y, test_size=0.5, random_state=123
)

def copy_to_split(file_list, label_list, split_name):
    for fp, lab in zip(file_list, label_list):
        dest = OUT_ROOT / split_name / lab
        dest.mkdir(parents=True, exist_ok=True)
        src = Path(fp)
        # handle collisions by adding suffix
        dst = dest / src.name
        if dst.exists():
            base, ext = os.path.splitext(src.name)
            i=1
```

```
  while (dest / f"{base}__dup{i}{ext}").exists():
            i+=1
        dst = dest / f"{base}__dup{i}{ext}"
    shutil.copy2(src, dst)

copy_to_split(train_files, train_y, "train")
copy_to_split(val_files, val_y, "val")
copy_to_split(test_files, test_y, "test")

# Print counts
for s in ["train","val","test"]:
  print(s, {lab: len(list((OUT_ROOT/s/lab).glob("*"))) for lab in labels})
```

11.4 Build tf.data datasets (with augmentation & preprocessing)

```
# STEP 2: build tf.data datasets
import tensorflow as tf
from tensorflow.keras import layers

SPLIT_DIR = "dataset/splits"
IMG_SIZE = (224,224)
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE
SEED = 123

train_ds = tf.keras.utils.image_dataset_from_directory(
  SPLIT_DIR + "/train",
  labels="inferred",
  label_mode="int",   # 0/1
  batch_size=BATCH_SIZE,
  image_size=IMG_SIZE,
  shuffle=True,
  seed=SEED
)

val_ds = tf.keras.utils.image_dataset_from_directory(
  SPLIT_DIR + "/val",
  labels="inferred",
  label_mode="int",
  batch_size=BATCH_SIZE,
  image_size=IMG_SIZE,
  shuffle=False
)
```

```
test_ds = tf.keras.utils.image_dataset_from_directory(
    SPLIT_DIR + "/test",
    labels="inferred",
    label_mode="int",
    batch_size=BATCH_SIZE,
    image_size=IMG_SIZE,
    shuffle=False
)

# Prefetch and caching
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)

# Data augmentation as a layer (applied on GPU when model trains)
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.08),
    layers.RandomTranslation(0.05, 0.05),
    layers.RandomContrast(0.08),
])
```

11.5 Build & compile model (EfficientNetB0 transfer learning)

```
# STEP 4: build model
from tensorflow.keras import layers, models
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input

def build_model(img_size=IMG_SIZE, num_classes=1, dropout_rate=0.3):
    inputs = layers.Input(shape=img_size + (3,))
    x = data_augmentation(inputs)
    x = preprocess_input(x)
    base = EfficientNetB0(weights="imagenet", include_top=False, input_tensor=x)
    base.trainable = False  # freeze initially
    x = layers.GlobalAveragePooling2D(name="gap")(base.output)
    x = layers.Dropout(dropout_rate)(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(0.2)(x)
    # binary classification: single unit + sigmoid
    outputs = layers.Dense(1, activation="sigmoid")(x)
```

```
    model = models.Model(inputs, outputs)

 return model

model = build_model()
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy", tf.keras.metrics.AUC(name="auc")]
)

model.summary()
```

11.6 Train with callbacks & optional fine-tuning

```
# STEP 5: training (head training, then fine-tune)
import datetime
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping,
ReduceLROnPlateau

LOG_DIR = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
ckpt = ModelCheckpoint("best_brain_tumour.h5", monitor="val_auc", mode="max",
save_best_only=True, verbose=1)
es = EarlyStopping(monitor="val_auc", mode="max", patience=8,
restore_best_weights=True)
rlr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3, verbose=1)

EPOCHS_HEAD = 12
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS_HEAD,
    class_weight=class_weight_dict,
    callbacks=[ckpt, es, rlr]
)

# Fine-tune: unfreeze last layers of base
base_model = model.get_layer(index=2)  # EfficientNetB0 base is second layer after Input and
augmentation+preproc; adapt if different
base_model.trainable = True

# Freeze early layers to avoid destroying learned features
fine_tune_at = int(len(base_model.layers) * 0.6)  # unfreeze last 40% layers
```

```
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="binary_crossentropy",
    metrics=["accuracy", tf.keras.metrics.AUC(name="auc")]
)

EPOCHS_FINE = 12
history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS_FINE,
    class_weight=class_weight_dict,
    callbacks=[ckpt, es, rlr]
)
```

11.7 A — Inspect progress & evaluate on test set

```
# 1) Plot training curves (use history and history_fine returned earlier)
import matplotlib.pyplot as plt
def plot_combined(hist1, hist2=None):
    # hist1 = history (head), hist2 = history_fine (fine-tune) — either may be None
    def plot(h, title_suffix=""):
        if h is None: return
        plt.figure(figsize=(12,4))
        plt.subplot(1,2,1)
        plt.plot(h.history.get("accuracy",[]), label="train_acc")
        plt.plot(h.history.get("val_accuracy",[]), label="val_acc")
        plt.legend(); plt.title("Accuracy " + title_suffix)
        plt.subplot(1,2,2)
        plt.plot(h.history.get("loss",[]), label="train_loss")
        plt.plot(h.history.get("val_loss",[]), label="val_loss")
        plt.legend(); plt.title("Loss " + title_suffix)
        plt.show()
    plot(hist1, "(head training)")
    plot(hist2, "(fine-tuning)")
# call this with the variables you used:
plot_combined(history if 'history' in globals() else None, history_fine if 'history_fine' in
globals() else None)
```

```
# 2) Evaluate on the test set (get final metrics)
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

# load best weights (your fine-tune saved best_brain_tumour.keras)
try:
    model.load_weights("best_brain_tumour.keras")
except Exception:
    print("Could not load .keras checkpoint; using current model weights.")

# predict on test set
y_true = np.concatenate([y.numpy() for x,y in test_ds], axis=0)
y_probs = model.predict(test_ds).ravel()
y_pred = (y_probs >= 0.5).astype(int)

print("Test ROC AUC:", roc_auc_score(y_true, y_probs))
print("Confusion matrix:")
print(confusion_matrix(y_true, y_pred))
print("Classification report:")
print(classification_report(y_true, y_pred, digits=4))
```

11.8 Save the final model properly (Keras format)

```
model.save("brain_tumour_classifier.keras")
print("Model saved as brain_tumour_classifier.keras")

import shutil

# Path to your model file in Colab
src_path = "/content/brain_tumour_classifier.keras"

# Destination folder in Drive
dst_path = "/content/drive/MyDrive/brain_tumour_classifier.keras"

# Copy file
shutil.copy(src_path, dst_path)

print("Model uploaded to Drive at:", dst_path)


import os
# Cell 2 — mount Drive and copy .keras model to local VM
from google.colab import drive
```

```
drive.mount('/content/drive')

# Edit this to your Drive model path:


DRIVE_MODEL_PATH = "/content/drive/MyDrive/brain_tumour_classifier.keras"  # <-
change if needed

# Destination on VM
LOCAL_MODEL_PATH = "/content/brain_tumour_classifier.keras"

# Copy if file exists
if os.path.exists(DRIVE_MODEL_PATH):
   import shutil
   shutil.copy(DRIVE_MODEL_PATH, LOCAL_MODEL_PATH)
   print("Copied model to", LOCAL_MODEL_PATH)
else:
   raise FileNotFoundError(f"Model not found in Drive at: {DRIVE_MODEL_PATH}. Check
path and try again.")
```

11.9 Load the model

```
from tensorflow.keras.models import load_model

# Cell 3 — load the Keras model
print("Loading model from", LOCAL_MODEL_PATH)
model = load_model(LOCAL_MODEL_PATH)
print("Model loaded. Summary:")
model.summary()
```

11.10 Upload a local image (dummy image) from your computer

```
# Cell 4 — upload an image from your local machine (dummy image)
from google.colab import files

print("Use the file picker to upload one image (jpg/png).")
uploaded = files.upload()  # choose file(s)
if len(uploaded) == 0:
   raise SystemExit("No file uploaded.")
# take the first uploaded file
uploaded_filename = list(uploaded.keys())[0]
uploaded_path = os.path.join("/content", uploaded_filename)
```

```
print("Uploaded to:", uploaded_path)
```

11.11 Prediction + Grad-CAM localization function

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image, ImageDraw
from skimage import measure
import tensorflow as tf
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.efficientnet import preprocess_input

# ---- SETTINGS ----
IMG_SIZE = (224, 224)
DECISION_THRESHOLD = 0.5
HEATMAP_THRESHOLD = 0.4
MIN_AREA = 50

def load_and_preprocess_pil(img_path):
pil = Image.open(img_path).convert("RGB")
arr224 = pil.resize(IMG_SIZE)
arr224 = np.array(arr224).astype(np.uint8)

inp = np.expand_dims(arr224.copy(), axis=0)
inp = preprocess_input(inp)

return pil, arr224, inp

def make_gradcam_heatmap_pil(img_array, model, last_conv_layer_name="top_conv",
eps=1e-8):
grad_model = tf.keras.models.Model(
[model.inputs],
[model.get_layer(last_conv_layer_name).output, model.output]
)

with tf.GradientTape() as tape:
conv_outputs, preds = grad_model(img_array)
loss = preds[:, 0]

grads = tape.gradient(loss, conv_outputs)
conv_outputs = conv_outputs[0].numpy()
grads = grads[0].numpy()
weights = np.mean(grads, axis=(0, 1))
```

```
heatmap = np.dot(conv_outputs, weights)

heatmap = np.maximum(heatmap, 0)

if heatmap.max() != 0:
heatmap /= heatmap.max()

return heatmap

def overlay_heatmap_pil(original_pil, heatmap):
heatmap_img = Image.fromarray(np.uint8(heatmap * 255))
heatmap_img = heatmap_img.resize(original_pil.size)
heatmap_img = heatmap_img.convert("RGB")

# Colorize using matplotlib colormap
import matplotlib.cm as cm
colormap = cm.get_cmap("jet")
colored = colormap(np.array(heatmap_img) / 255.0)[:, :, :3]
colored = (colored * 255).astype(np.uint8)
colored_img = Image.fromarray(colored)

blended = Image.blend(original_pil, colored_img, alpha=0.5)
return blended

def bbox_from_heatmap_pil(heatmap_resized):
mask = (heatmap_resized >= HEATMAP_THRESHOLD).astype(np.uint8)
labels = measure.label(mask, connectivity=2)

if labels.max() == 0:
return None, mask

props = measure.regionprops(labels)
areas = [p.area for p in props]
max_idx = np.argmax(areas)

if areas[max_idx] < MIN_AREA:
return None, mask

y_min, x_min, y_max, x_max = props[max_idx].bbox
return (x_min, y_min, x_max, y_max), mask

def predict_and_localize_no_cv(img_path, model, last_conv="top_conv", display=True):
original_pil, arr224, inp = load_and_preprocess_pil(img_path)
```

```
prob = float(model.predict(inp)[0][0])
label = "Tumour" if prob >= DECISION_THRESHOLD else "No Tumour"

print(f"Prediction: {label} (prob={prob:.4f})")

if label == "No Tumour":
plt.imshow(original_pil)
plt.title(f"No Tumour (prob={prob:.4f})")
plt.axis("off")
return

# Grad-CAM
heatmap_low = make_gradcam_heatmap_pil(inp, model, last_conv)
heatmap_resized = np.array(Image.fromarray((heatmap_low *
255).astype(np.uint8)).resize(original_pil.size)) / 255.0

overlay = overlay_heatmap_pil(original_pil, heatmap_resized)

bbox, mask = bbox_from_heatmap_pil(heatmap_resized)

overlay_box = overlay.copy()
if bbox:
draw = ImageDraw.Draw(overlay_box)
x_min, y_min, x_max, y_max = bbox
draw.rectangle([x_min, y_min, x_max, y_max], outline="yellow", width=3)

# Display results
if display:
plt.figure(figsize=(14, 6))
plt.subplot(1, 3, 1); plt.imshow(original_pil); plt.title("Original"); plt.axis("off")
plt.subplot(1, 3, 2); plt.imshow(overlay); plt.title("Grad-CAM Overlay"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(overlay_box); plt.title(f"Detected Tumour {bbox}");
plt.axis("off")
plt.show()

return {"prob": prob, "label": label, "bbox": bbox, "overlay": overlay_box}
```

11.12 Run the prediction & localization on the uploaded image

```
# Fixed no-OpenCV Grad-CAM + overlay (replacement cell)
import numpy as np
import matplotlib.pyplot as plt
```

```
from PIL import Image, ImageDraw
from skimage import measure
import tensorflow as tf
from tensorflow.keras.preprocessing import image

from tensorflow.keras.applications.efficientnet import preprocess_input

# settings
IMG_SIZE = (224,224)
DECISION_THRESHOLD = 0.5
HEATMAP_THRESHOLD = 0.4
MIN_AREA = 50

def load_and_preprocess_pil(img_path):
    original_pil = Image.open(img_path).convert("RGB")
    resized = original_pil.resize(IMG_SIZE)
    arr224 = np.array(resized).astype(np.uint8)
    inp = np.expand_dims(arr224.copy(), axis=0)
    inp = preprocess_input(inp)
    return original_pil, arr224, inp

def make_gradcam_heatmap_pil(img_array, model, last_conv_layer_name="top_conv",
eps=1e-8):
    grad_model = tf.keras.models.Model([model.inputs],
[model.get_layer(last_conv_layer_name).output, model.output])
    with tf.GradientTape() as tape:
        conv_outputs, preds = grad_model(img_array)
        loss = preds[:, 0]
    grads = tape.gradient(loss, conv_outputs)
    conv_outputs = conv_outputs[0].numpy()   # H x W x C
    grads = grads[0].numpy()                 # H x W x C
    weights = np.mean(grads, axis=(0,1))     # C
    # weighted sum of feature maps
    heatmap = np.tensordot(conv_outputs, weights, axes=([2],[0]))  # H x W
    heatmap = np.maximum(heatmap, 0)
    if heatmap.max() != 0:
        heatmap = heatmap / (heatmap.max() + eps)
    return heatmap

def overlay_heatmap_pil(original_pil, heatmap_2d, alpha=0.5, cmap_name="jet"):
    # heatmap_2d expected 2D float 0..1 with same spatial size as original_pil when resized
below
    # Resize heatmap to original size
```

```
    heatmap_img = Image.fromarray(np.uint8(heatmap_2d * 255))          # 2D -> grayscale
image
    heatmap_resized = heatmap_img.resize(original_pil.size, resample=Image.BILINEAR)
    heatmap_resized_arr = np.array(heatmap_resized) / 255.0            # 2D float 0..1
    # Apply colormap (matplotlib) on 2D heatmap
    cmap = plt.get_cmap(cmap_name)
    colored = cmap(heatmap_resized_arr)[:, :, :3]                      # H x W x 3 float 0..1
    colored_uint8 = (colored * 255).astype(np.uint8)
    colored_img = Image.fromarray(colored_uint8)
    # Blend with original
    blended = Image.blend(original_pil.convert("RGBA"), colored_img.convert("RGBA"),
alpha=alpha)
    return blended.convert("RGB"), heatmap_resized_arr

def bbox_from_heatmap_pil(heatmap_resized_arr, threshold=HEATMAP_THRESHOLD,
min_area=MIN_AREA):
    mask = (heatmap_resized_arr >= threshold).astype(np.uint8)
    labels = measure.label(mask, connectivity=2)
    if labels.max() == 0:
        return None, mask
    props = measure.regionprops(labels)
    areas = [p.area for p in props]
    max_idx = int(np.argmax(areas))
    if areas[max_idx] < min_area:
        return None, mask
    y_min, x_min, y_max, x_max = props[max_idx].bbox
    return (x_min, y_min, x_max, y_max), mask

def predict_and_localize_no_cv_fixed(img_path, model, last_conv="top_conv", display=True,
save_overlay_path=None):
    original_pil, arr224, inp = load_and_preprocess_pil(img_path)
    prob = float(model.predict(inp)[0][0])
    label = "Tumour" if prob >= DECISION_THRESHOLD else "No Tumour"
    print(f"Prediction: {label} (prob={prob:.4f})")
    if label == "No Tumour":
        if display:
            plt.figure(figsize=(6,6)); plt.imshow(original_pil); plt.title(f"No Tumour
(prob={prob:.4f})"); plt.axis("off"); plt.show()
        return {"label": label, "probability": prob, "bbox": None, "overlay": None}
    # Grad-CAM heatmap (low-res)
    heatmap_low = make_gradcam_heatmap_pil(inp, model, last_conv)
    # Resize heatmap to original image size and create overlay
    overlay_img, heatmap_resized_arr = overlay_heatmap_pil(original_pil, heatmap_low,
alpha=0.5, cmap_name="jet")
```

```
    bbox, mask = bbox_from_heatmap_pil(heatmap_resized_arr)
    # draw bbox on overlay copy
    overlay_with_box = overlay_img.copy()
    if bbox is not None:
        draw = ImageDraw.Draw(overlay_with_box)
        x_min, y_min, x_max, y_max = bbox
        draw.rectangle([x_min, y_min, x_max, y_max], outline=(255,255,0), width=3)

  if display:
        plt.figure(figsize=(14,6))
        plt.subplot(1,3,1); plt.imshow(original_pil); plt.title("Original"); plt.axis("off")
        plt.subplot(1,3,2); plt.imshow(overlay_img); plt.title(f"Grad-CAM Overlay
(prob={prob:.3f})"); plt.axis("off")
        plt.subplot(1,3,3);
        if bbox is not None:
            plt.imshow(overlay_with_box); plt.title(f"Detected bbox: {bbox}")
        else:
            plt.imshow(overlay_img); plt.title("No region above threshold")
        plt.axis("off")
        plt.show()
    if save_overlay_path:
        overlay_with_box.save(save_overlay_path)
        print("Saved overlay to:", save_overlay_path)
    return {"label": label, "probability": prob, "bbox": bbox, "overlay": overlay_with_box,
"heatmap": heatmap_resized_arr}

# Run on the uploaded image (replace uploaded_path with your image path if needed)
res = predict_and_localize_no_cv_fixed(uploaded_path, model, last_conv="top_conv",
display=True, save_overlay_path="/content/overlay_fixed.png")
print("Result:", {k: (v if k!="heatmap" else "heatmap_array") for k,v in res.items()})
```

11.13 gradio with ouput image and prescribtics analytics

```
# Gradio app — prescriptive analytics (Markdown only, NO LLM summary)
!pip install -q gradio scikit-image imageio

import gradio as gr
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.efficientnet import preprocess_input
from PIL import Image, ImageDraw
from skimage import measure
```

```
import matplotlib.pyplot as plt
import os, json

# ---------- CONFIG ----------
LOCAL_MODEL_PATH = "/content/brain_tumour_classifier.keras"
IMG_SIZE = (224,224)
DECISION_THRESHOLD = 0.5
HEATMAP_THRESHOLD = 0.4

MIN_AREA = 50
LAST_CONV_NAME = "top_conv"

# ---------- Load model ----------
if not os.path.exists(LOCAL_MODEL_PATH):
    raise FileNotFoundError("Model file not found at path:", LOCAL_MODEL_PATH)

model = load_model(LOCAL_MODEL_PATH)
print("Model loaded:", LOCAL_MODEL_PATH)

# ---------- Helpers ----------
def pil_to_arr224(pil_img):
    pil = pil_img.convert("RGB")
    resized = pil.resize(IMG_SIZE)
    arr = np.array(resized).astype(np.uint8)
    return pil, arr

def preprocess_for_model(arr):
    x = np.expand_dims(arr.copy(), axis=0)
    x = preprocess_input(x)
    return x

def make_gradcam_heatmap_pil(img_tensor, model, last_conv_layer_name, eps=1e-8):
    grad_model = tf.keras.models.Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )
    with tf.GradientTape() as tape:
        conv_out, preds = grad_model(img_tensor)
        loss = preds[:, 0]
    grads = tape.gradient(loss, conv_out)
    conv_out = conv_out[0].numpy()
    grads = grads[0].numpy()
    weights = np.mean(grads, axis=(0,1))
    heatmap = np.tensordot(conv_out, weights, axes=([2],[0]))
```

```
    heatmap = np.maximum(heatmap, 0)
    if heatmap.max() != 0:
        heatmap /= heatmap.max() + eps
    return heatmap

def overlay_heatmap_pil(original_pil, heatmap_2d, alpha=0.5):
    heatmap_img = Image.fromarray(np.uint8(heatmap_2d * 255))
    resized = heatmap_img.resize(original_pil.size)
    arr = np.array(resized) / 255.0

    cmap = plt.get_cmap("jet")
    colored = cmap(arr)[:, :, :3]
    colored_uint8 = (colored * 255).astype(np.uint8)

    overlay = Image.blend(
        original_pil.convert("RGBA"),
        Image.fromarray(colored_uint8).convert("RGBA"),
        alpha
    )
    return overlay.convert("RGB"), arr

def bbox_from_heatmap(heatmap_arr, thr=HEATMAP_THRESHOLD,
min_area=MIN_AREA):
    mask = (heatmap_arr >= thr).astype(np.uint8)
    labels = measure.label(mask, connectivity=2)
    if labels.max() == 0:
        return None, mask
    props = measure.regionprops(labels)
    areas = [p.area for p in props]
    idx = int(np.argmax(areas))
    if areas[idx] < min_area:
        return None, mask
    y1, x1, y2, x2 = props[idx].bbox
    return (x1, y1, x2, y2), mask

def map_bbox_to_brain_region(bbox, image_size):
    if bbox is None:
        return "Unknown"
    x1, y1, x2, y2 = bbox
    cx = (x1 + x2) / 2
    cy = (y1 + y2) / 2

    W, H = image_size
    nx, ny = cx / W, cy / H
```

```
    side = "Left hemisphere" if nx < 0.5 else "Right hemisphere"

    if ny < 0.35:
        lobe = "Frontal lobe"
    elif ny < 0.65:
        if 0.25 < nx < 0.75:
            lobe = "Parietal lobe"
        else:
            lobe = "Temporal lobe"

    else:
        lobe = "Occipital lobe"

    return f"{lobe}, {side}"

def prescriptive_dict_to_markdown(info):
    prob = info["probability"]
    region = info["region"]

    md = f"""
## Brain Tumour Summary

**Prediction:** Tumour
**Probability:** **{prob*100:.2f}%**

---

### Approximate location
**{region}**

> This is an automated estimate based on heatmap localization.
> Always confirm the exact location using full MRI sequences + radiologist review.
"""
    return md

# ---------- Main prediction ----------
def predict_and_prescribe_md(pil_img):
    orig, arr = pil_to_arr224(pil_img)
    x = preprocess_for_model(arr)
    prob = float(model.predict(x)[0][0])
    label = "Tumour" if prob >= DECISION_THRESHOLD else "No Tumour"

    overlay_img = orig.copy()
```

```
    state = {"probability": prob, "region": None, "detected": label=="Tumour"}
    prescriptive_md = ""

    if label == "Tumour":
        heatmap = make_gradcam_heatmap_pil(x, model, LAST_CONV_NAME)
        overlay, hres = overlay_heatmap_pil(orig, heatmap)
        bbox, _ = bbox_from_heatmap(hres)

        if bbox:
            draw = ImageDraw.Draw(overlay)
            x1,y1,x2,y2 = bbox

     draw.rectangle([x1,y1,x2,y2], outline=(255,255,0), width=3)

        overlay_img = overlay

        region = map_bbox_to_brain_region(bbox, orig.size)
        state["region"] = region

        prescriptive_md = prescriptive_dict_to_markdown({
            "probability": prob,
            "region": region
        })

    else:
        draw = ImageDraw.Draw(overlay_img)
        draw.text((10,10), f"No Tumour ({prob:.3f})", fill=(255,0,0))
        prescriptive_md = f"### No Tumour Detected\nProbability: {prob:.3f}"

    return f"{label} (prob={prob:.4f})", prob, overlay_img, prescriptive_md, state

# ---------- Dropdown detail ----------
def dropdown_detail(choice, state):
    if not state:
        return "Run a prediction first."

    if not state["detected"]:
        return "No tumour detected — no additional details available."

    prob = state["probability"]
    region = state["region"]

    if choice == "Probability of tumour":
        return f"""
```

```
### Probability of Tumour
**{prob*100:.2f}%**

This value reflects how confident the model is that the MRI contains a tumour-like
abnormality.
"""

    if choice == "Precise tumour location":
        return f"""
### Tumour Location (Estimated)
**{region}**

This is an approximate region based on Grad-CAM.

Exact location must be confirmed with detailed MRI review.
"""

    return "Select an option from the dropdown."

# ---------- UI ----------
with gr.Blocks() as demo:
    gr.Markdown("## Brain Tumour Detection + Minimal Prescriptive Analysis")

    with gr.Row():
        img_in = gr.Image(type="pil", label="Upload MRI")
        with gr.Column():
            out_label = gr.Textbox(label="Prediction")
            out_prob  = gr.Number(label="Probability")
            out_img   = gr.Image(type="pil", label="Overlay (Grad-CAM)")
            out_md    = gr.Markdown(label="Summary Report")

    state = gr.State()

    detail_dd = gr.Dropdown(
        label="More Details",
        choices=["Select...", "Probability of tumour", "Precise tumour location"],
        value="Select..."
    )
    detail_md = gr.Markdown()

    btn = gr.Button("Predict")

    btn.click(
        fn=predict_and_prescribe_md,
```

```
        inputs=img_in,
        outputs=[out_label, out_prob, out_img, out_md, state]
    )

    detail_dd.change(
        fn=dropdown_detail,
        inputs=[detail_dd, state],
        outputs=detail_md
    )

demo.launch(share=True)
```